

Growth Intern Assignment

Jar — Submission

Submitted by: Ganesha S

Email: ganesh849894@gmail.com

Date: 22 July 2026

Code repository: <https://github.com/Ganesh-anandam/jar-assesment>

This submission is my own original work, based on the datasets and the Jar app provided as part of this assignment. The full Python code, notebook and commit history are available at the repository link above for review.

Question 1 — Sales Analysis

The three files provided – `List_of_Orders.xlsx`, `Order_Details.xlsx` and `Sales_target.xlsx` – cover the order-level details, the line-item level category/profit breakdown, and the monthly category targets respectively. All of the work below is done in Python (pandas for the analysis, matplotlib for the charts). I've kept the data-cleaning steps in, since two of the columns needed genuine fixing before the numbers could be trusted.

```
import pandas as pd
import matplotlib.pyplot as plt

orders = pd.read_excel('List_of_Orders.xlsx')
details = pd.read_excel('Order_Details.xlsx')
target = pd.read_excel('Sales_target.xlsx')
```

A couple of data quality issues, found before doing any analysis

Order Date in `List_of_Orders.xlsx` turned out to be a mix of real date objects and plain dd-mm-yyyy text (about 307 of the 500 rows came in as text, e.g. 13-04-2018). Calling `pd.to_datetime()` without `dayfirst=True` would silently mis-parse those rows.

The month column in `Sales_target.xlsx` was worse: it was originally typed as Apr-18, May-18 etc., but Excel's autocorrect turned that text into a full date – and used the day field to store the two-digit year (18/19) instead of an actual day. The month is trustworthy, the day is really a year suffix, and the year Excel assigned on top of that is fake and identical across every row. So the label has to be rebuilt from the month name plus that stray day value, and any sorting has to use that reconstructed pair, not the raw date column.

```
orders['Order Date'] = pd.to_datetime(orders['Order Date'], dayfirst=True)

target['year_suffix'] = target['Month of Order Date'].dt.day
target['month_num'] = target['Month of Order Date'].dt.month
target['month_label'] = (
    target['Month of Order Date'].dt.strftime('%b') + '-' +
    target['year_suffix'].astype(str)
)
```

Both issues are confirmed against the raw data rather than assumed – e.g. text rows like 13-04-2018 can only be dd-mm-yyyy, since 13 can't be a month.

Part 1: Sales and Profitability Analysis

Merging orders with details on Order ID gives one row per line item, with state/city/date attached. Every ID matches on both sides (checked below), so an inner join doesn't drop or duplicate

anything.

```
df = orders.merge(details, on='Order ID', how='inner')

# a plain .mean() on Profit would overstate "average profit per order": 322 of 783
# (Category, Order ID) combinations have more than one line item, so collapse to
# one row per order first, then average across the distinct orders
profit_per_order = df.groupby(['Category', 'Order ID'])
['Profit'].sum().groupby('Category').mean()

category_summary = df.groupby('Category').agg(
    total_sales=('Amount', 'sum'),
    total_profit=('Profit', 'sum'),
).sort_values('total_sales', ascending=False)

category_summary['distinct_orders'] = df.groupby('Category')['Order ID'].nunique()
category_summary['avg_profit_per_order'] = profit_per_order
category_summary['profit_margin_pct'] = category_summary['total_profit'] /
category_summary['total_sales'] * 100
```

Category	Total Sales	Total Profit	Distinct Orders	Avg Profit / Order	Margin %
Electronics	165,267	10,494	204	51.44	6.35%
Clothing	139,054	11,163	393	28.40	8.03%
Furniture	127,181	2,298	186	12.35	1.81%

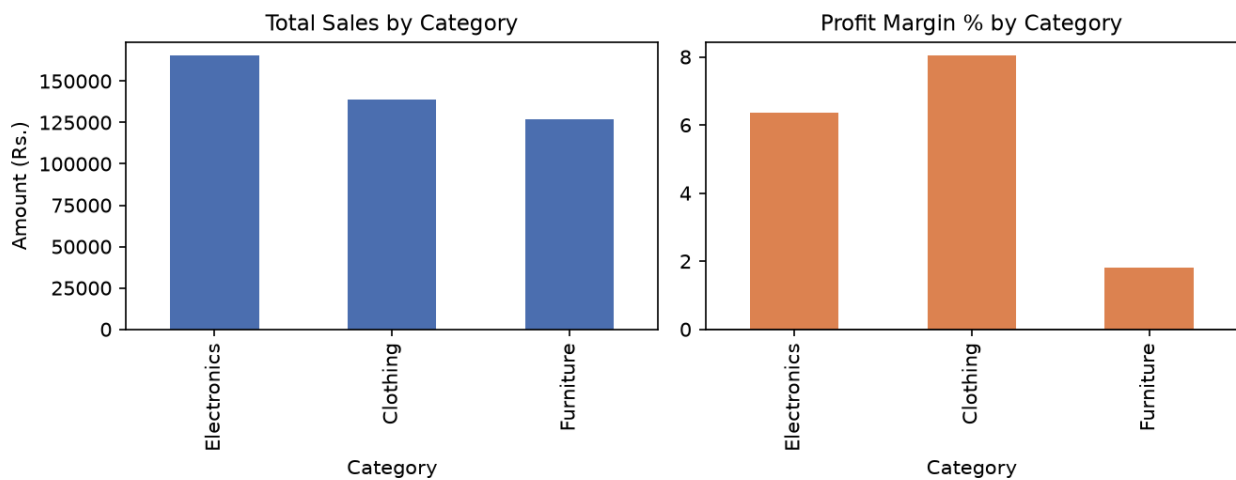


Figure 1 - Total sales and profit margin by category

Sales alone don't explain the margin gap, so breaking each category down by sub-category shows where the profit is actually being made or lost:

```
sub_summary = df.groupby(['Category', 'Sub-Category']).agg(
    sales=('Amount', 'sum'), profit=('Profit', 'sum'), orders=('Order ID', 'count'),
).reset_index()
sub_summary['margin_pct'] = sub_summary['profit'] / sub_summary['sales'] * 100
```

Category	Sub-Category	Sales	Profit	Orders	Margin %
Clothing	Kurti	3,361	181	47	5.39%
Clothing	Skirt	1,946	235	64	12.08%
Clothing	Leggings	2,106	260	53	12.35%
Clothing	Saree	53,511	352	210	0.66%
Clothing	Shirt	7,555	1,131	69	14.97%
Clothing	T-shirt	7,382	1,500	77	20.32%
Clothing	Hankerchief	14,608	2,098	198	14.36%
Clothing	Stole	18,546	2,559	192	13.80%
Clothing	Trousers	30,039	2,847	39	9.48%
Electronics	Electronic Games	39,168	-1,236	79	-3.16%
Electronics	Phones	46,119	2,207	83	4.79%
Electronics	Accessories	21,728	3,559	72	16.38%
Electronics	Printers	58,252	5,964	74	10.24%
Furniture	Tables	22,614	-4,011	17	-17.74%
Furniture	Chairs	34,222	577	74	1.69%
Furniture	Furnishings	13,484	844	73	6.26%
Furniture	Bookcases	56,861	4,888	79	8.60%

Findings

Electronics is the biggest category by revenue (Rs. 1.65L) and by far the best on a per-order basis – an order that includes an Electronics item nets about Rs. 51 profit on average, roughly 1.8× Clothing (Rs. 28) and over 4× Furniture (Rs. 12). That fits a higher-ticket-size category (Printers, Phones) where a handful of profitable orders carry the numbers. The one soft spot is Electronic Games, which is loss-making overall (-3.2% margin) and drags the category average down slightly.

Clothing has the healthiest margin of the three (~8.0%) and gets there without a single loss-making sub-category – Sarees make up the bulk of the revenue but at a thin margin (0.66%), while Stole, Hankerchief and T-shirt are what actually carry the profit. It also reaches the most orders by a distance (393 distinct orders touch Clothing vs. 204 for Electronics and 186 for Furniture), so consistency across volume matters more here than any single big win.

Furniture is the clear underperformer – lowest total profit (Rs. 2,298), lowest margin (1.8%), and lowest average profit per order (Rs. 12.35). 42% of Furniture line items lose money, the highest loss-rate of any category, and it is driven almost entirely by Tables, sitting at a -17.7% margin on its own. Bookcases and Furnishings are fine, so this is a Tables-specific problem rather than the whole category being broken – heavier, bulkier items typically carry higher shipping, handling and return costs relative

to sale price, which is consistent with a loss of this size. Furniture is also the category the assignment asks about for target trends in Part 2, which lines up with how weak these numbers already look.

Part 2: Target Achievement Analysis (Furniture)

Sales_target.xlsx only has one row per (month, category) – there's no actual sales figure to compare against, so this part is about the target trend itself: how fast the bar is being raised, and whether that pace looks sensible.

```
furniture_target = (
    target[target['Category'] == 'Furniture']
    # NOT .sort_values('Month of Order Date') - that column's year is fake and
    # identical across every row, so sorting on it would put Jan/Feb/Mar-19 before
    # Apr-18 instead of after Dec-18
    .sort_values(['year_suffix', 'month_num'])
    .reset_index(drop=True)
)
furniture_target['mom_change_pct'] = furniture_target['Target'].pct_change() * 100
```

Month	Target	MoM Change %
Apr-18	10,400	–
May-18	10,500	0.96%
Jun-18	10,600	0.95%
Jul-18	10,800	1.89%
Aug-18	10,900	0.93%
Sep-18	11,000	0.92%
Oct-18	11,100	0.91%
Nov-18	11,300	1.80%
Dec-18	11,400	0.88%
Jan-19	11,500	0.88%
Feb-19	11,600	0.87%
Mar-19	11,800	1.72%

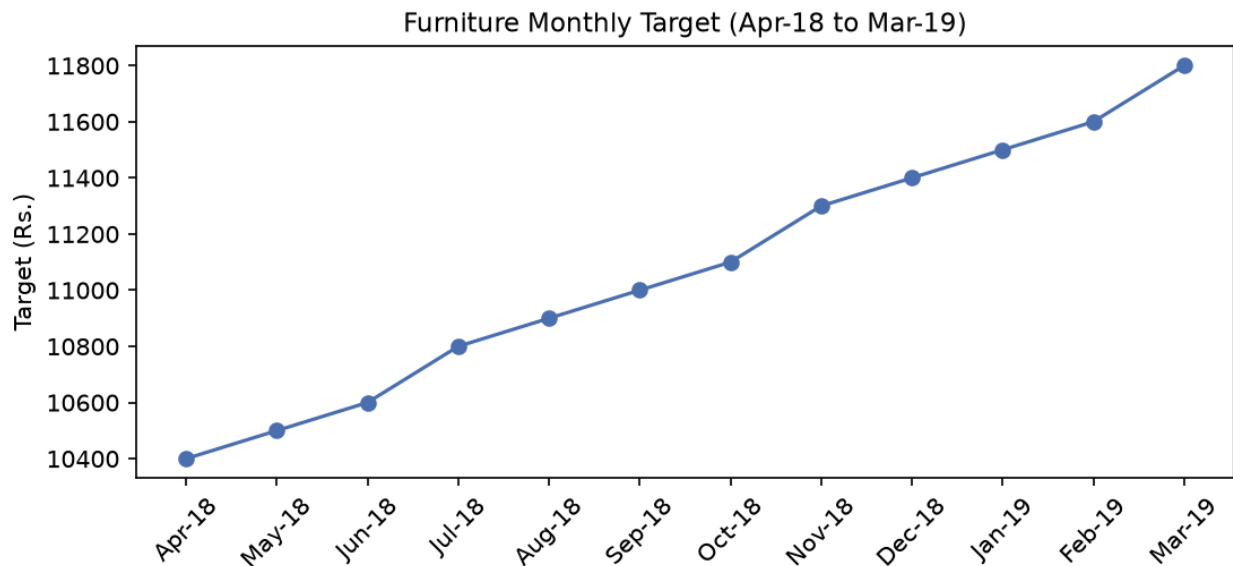


Figure 2 - Furniture monthly target, Apr-18 to Mar-19

Findings

The target climbs every month – there's no month where it drops – but the pace isn't constant. Most months step up by a modest $\sim 0.9\%$, and three months (**Jul-18**, **Nov-18** and **Mar-19**) jump by roughly $1.7\text{--}1.9\%$, about double the usual increase. The +4-month spacing (Jul $\rightarrow$ Nov $\rightarrow$ Mar) is a clean pattern rather than noise, so it looks like whoever set these targets built in a bigger quarterly push on top of steady monthly growth.

That's a reasonable planning habit on its own, but it's worth pairing against Part 1: Furniture is already the weakest-margin, highest-loss-rate category, and a chunk of its revenue (Tables specifically) is sold at a loss. Raising the target steadily assumes profitability keeps up, which isn't guaranteed here. Two concrete suggestions: (1) set targets per sub-category rather than per category – Bookcases and Furnishings can likely absorb steady growth, while Tables needs a target tied to fixing margin, not just moving more units; (2) re-baseline the quarterly step-ups against actual achievement rather than the previous target, since compounding $\sim 1\%$ growth (plus a bump every quarter) on top of a category that's already missing target just widens the gap each period.

Part 3: Regional Performance Insights

This part only needs orders for the state/city and order count, joined back to `details` for sales and profit.

```
top5_states = orders['State'].value_counts().head(5)

state_rows = df[df['State'].isin(top5_states.index)]
avg_profit_per_order = (
    state_rows.groupby(['State', 'Order ID'])['Profit'].sum().groupby('State').mean()
)
state_summary = state_rows.groupby('State').agg(
```

```

total_sales=('Amount', 'sum'), total_profit=('Profit', 'sum'),
)
state_summary['orders']          = top5_states # true order count, not a line-
item count
state_summary['avg_profit_per_order'] = avg_profit_per_order
state_summary['margin_pct']         = state_summary['total_profit'] /
state_summary['total_sales'] * 100

```

State	Orders	Total Sales	Avg Profit / Order	Total Profit	Margin %
Madhya Pradesh	101	105,140	54.96	5,551	5.28%
Maharashtra	90	95,348	68.62	6,176	6.48%
Rajasthan	32	21,149	39.28	1,257	5.94%
Gujarat	27	21,058	17.22	465	2.21%
Punjab	25	16,786	-24.36	-609	-3.63%

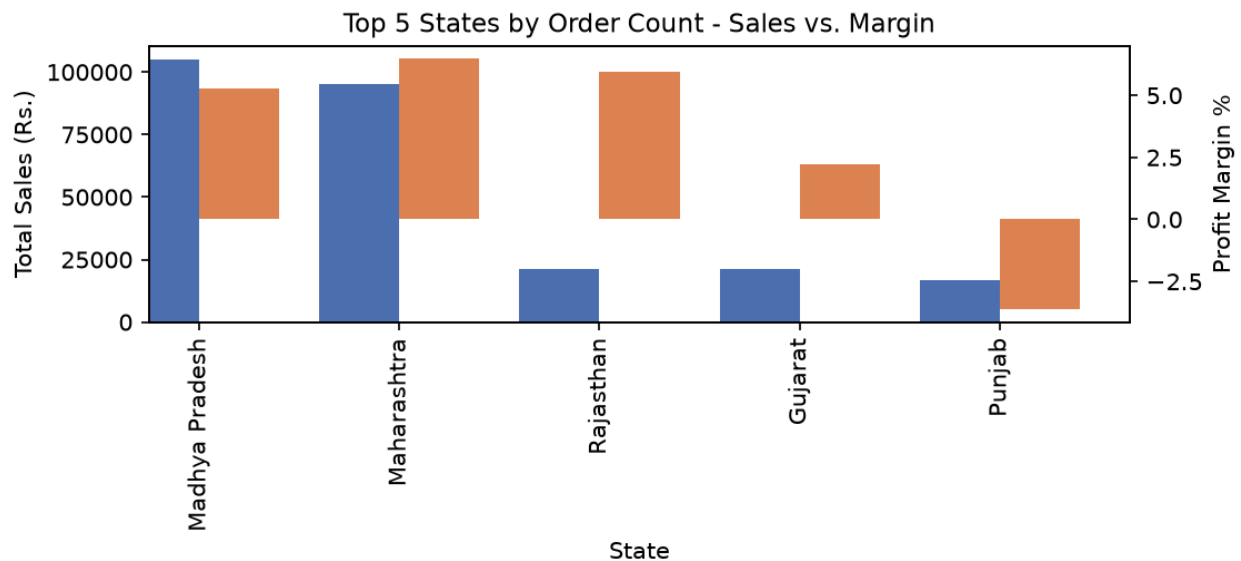


Figure 3 - Top 5 states by order count: sales vs. margin

The top-5-by-orders list and the sales figures don't move together, so it's worth checking margin across all states, and then drilling into cities within the top 5:

```

all_state_summary = df.groupby('State').agg(total_sales=('Amount', 'sum'),
total_profit=('Profit', 'sum'))
all_state_summary['orders']      = orders['State'].value_counts()
all_state_summary['margin_pct'] = all_state_summary['total_profit'] /
all_state_summary['total_sales'] * 100

city_summary = state_rows.groupby('City').agg(total_sales=('Amount', 'sum'),
total_profit=('Profit', 'sum'))
city_summary['orders']      = orders[orders['State'].isin(top5_states.index)]
['City'].value_counts()

```



```
city_summary['margin_pct'] = city_summary['total_profit'] / city_summary['total_sales']
* 100
```

States with the weakest margin (all states, not just the top 5):

State	Orders	Total Sales	Total Profit	Margin %
Tamil Nadu	8	6,087	-2,216	-36.41%
Andhra Pradesh	15	13,256	-496	-3.74%
Punjab	25	16,786	-609	-3.63%
Bihar	16	12,943	-321	-2.48%
Jammu and Kashmir	14	10,829	8	0.07%
Nagaland	15	11,903	148	1.24%

Cities within the top-5 states, by order count:

City	Orders	Total Sales	Total Profit	Margin %
Indore	76	79,069	4,159	5.26%
Mumbai	68	61,867	1,637	2.65%
Bhopal	22	23,583	871	3.69%
Pune	22	33,481	4,539	13.56%
Jaipur	19	10,076	-753	-7.47%
Ahmedabad	17	14,230	-880	-6.18%
Chandigarh	16	12,279	-1,153	-9.39%
Udaipur	13	11,073	2,010	18.15%
Surat	10	6,828	1,345	19.70%
Amritsar	9	4,507	544	12.07%
Delhi	3	2,488	521	20.94%

Findings

Madhya Pradesh and Maharashtra dominate order volume (101 and 90 orders – more than 3× the next closest state), and unsurprisingly they also produce the most total sales and profit in absolute terms. The real story is the 5th state:

- **Punjab is the standout problem among the top 5** – 25 orders, but a -3.6% margin and -Rs. 24 average profit per order, the only one of the five actually losing money. It's driven almost entirely by **Chandigarh** (-9.4% margin, 16 of Punjab's 25 orders) – Amritsar, the other city in the state, is healthy on its own (+12.1% margin), so this is a Chandigarh-specific issue, not a state-wide one.

- **Gujarat is also weaker than it looks** – 27 orders but only a 2.2% margin, well behind Maharashtra (6.5%) and Madhya Pradesh (5.3%). Almost all of that comes down to **Ahmedabad** (-6.2% margin), even though **Surat** in the same state is doing well (+19.7%).
- **Rajasthan tells a similar story at the city level** – the state overall sits at a reasonable 5.9% margin, but that's a strongly negative Jaipur (-7.5%) and a strongly positive Udaipur (+18.2%) roughly cancelling out.
- Zooming out past the top 5, **Punjab isn't even the worst state overall** – Tamil Nadu (-36.4%, though only 8 orders) and Andhra Pradesh (-3.7%) are both worse on margin. Low order counts mean these two won't move company-wide numbers much, but a -36% margin at any scale is unusual enough to be worth a root-cause look.

Where I'd prioritize: Punjab first – it's the only top-5-by-volume state actually losing money, so a fix (almost certainly something specific to Chandigarh – pricing, discounting, or return rates) has more leverage than the same fix would in a low-volume state. Ahmedabad and Jaipur are next, since they're negative-margin cities sitting inside states with decent volume and a healthy twin city right next to them, suggesting a local execution problem rather than the whole region being unviable. Tamil Nadu is the most extreme number on the sheet and probably deserves a look too, even at low volume, since -36% is too large to be normal variance.

Across all three parts, the pattern repeats: the "total sales" number and the "actually profitable" number disagree most in a few specific slices – Furniture (particularly Tables), and Chandigarh/Ahmedabad/Jaipur within otherwise-solid states. None of these are volume problems, they are margin problems, so the fix is more likely to be pricing, discounting or logistics for those specific slices rather than driving more orders to them.

Question 2 — App Exploration

I spent time going through the Jar app on a personal account before writing this - the save flows, the redemption side, the referral/gamification bits, and the settings/KYC screens. Below are the five things that stood out as genuinely well done, and five places where I'd push back or improve things, with the reasoning behind each.

Five things that work well

1. **Round-off saving removes the "decision" from saving.** Instead of asking the user to remember to save, it hooks into UPI transactions and invests the spare change automatically. The whole reason most people don't save small amounts consistently is that it requires a repeated, conscious decision - Jar just deletes that step. This is the single feature that made the app feel different from a regular gold-buying app rather than a "nice-to-have" add-on.
2. **₹10 minimum ticket size actually lowers the psychological barrier, not just the financial one.** A lot of savings products advertise "start small" but still make ₹500 feel like the real entry point. Being able to buy in single-digit rupee amounts means a first-time user can try the product without it feeling like a financial commitment, which matters a lot for people who've never invested in anything before.
3. **The streak/gamification layer gives saving a feedback loop.** Daily save streaks, small visual rewards, and progress indicators borrow directly from habit-forming app design (the kind you'd see in a language-learning app), and it works because saving money otherwise has almost no immediate feedback - you don't "feel" ₹20 going into gold the way you feel a streak counter going up.
4. **Redemption flexibility is more thought-through than I expected.** Being able to cash out, or alternatively convert savings into actual gold coins/jewellery through partner jewellers, means the product doesn't force a single exit path. For a user who started saving without a clear goal in mind, having options at the end rather than only at the start is a good design call.
5. **Trust signals around the gold itself are handled well.** 24K certified gold, clear custody/storage messaging, and transparent real-time pricing address the most obvious objection a first-time user would have ("is this actually backed by real gold, or is it just a number in an app?"). Getting this reassurance in early, rather than burying it in T&Cs, is probably doing a lot of quiet work on conversion.

Five areas I'd improve

1. **The connection between "saving" and "investing" isn't explained clearly enough inside the flow itself.** A first-time user rounding off their chai payment may not fully register that they're buying a commodity whose price moves daily, gold. A short, one-time explainer woven into the round-off setup (not a separate help article) would set expectations better and reduce confusion later when the redemption value isn't exactly what they'd expect.

2. **There's no visible way to compare gold savings against a benchmark or alternative.** Someone using the app for months has no easy way to see "here's what this would look like against a recurring deposit" or similar. Even a simple, non-prescriptive comparison view (not advice, just information) would help retention for users deciding whether to keep going or move the money elsewhere.
 3. **Goal-based saving feels bolted on rather than central.** The core loop is very good at "save a little, automatically", but less good at helping someone plan toward a specific outcome (a wedding, a gift, a specific gram target) with a visible countdown or milestone structure. Given how much of the emotional appeal of gold-buying in India is tied to specific occasions, this feels like an underused hook.
 4. **Notifications lean more toward re-engagement nudges than genuinely useful updates.** A user checking in after a price swing or a milestone in their own savings is a much stronger moment to message them than a generic "come back and save today" push. The current mix felt weighted toward the latter.
 5. **The onboarding/KYC step, while necessary, is where a chunk of curious first-time users will likely drop off,** since it's a harder ask than the ₹10 entry price suggests. Letting a user save a very small amount in an unverified "preview" state before asking for full KYC (converting to real gold only after verification) could let people experience the product before being asked to commit personal documents - right now those two asks happen almost back-to-back.
-

Question 3 — Product Exploration: New Business Opportunities

Jar's actual asset isn't gold - it's the habit it's built and the trust that comes with being the thing people let auto-debit their account every day. Digital gold was the right first product to earn that trust because it's simple to understand, low-risk in the way most Indian households already think about gold, and works at a ticket size nothing else in fintech was offering. But the same rails - automation, tiny ticket sizes, and a user base that already trusts Jar with recurring small debits - can carry more than one product. A few directions I think make sense, roughly in order of how close they are to what Jar already does:

1. Broader micro-investing beyond gold. The obvious next step is extending the same round-off/auto-save mechanism to other asset classes - short-duration debt funds, index funds, or even fractional silver - for users who've built a saving habit and are ready to diversify. The point isn't to compete head-on with mutual fund apps on features; it's to let the existing round-off engine feed into more than one destination, with gold as the default and other options as an upgrade path for users who stick around. This keeps the automation-first identity intact while raising the ceiling on what a long-term user can do with the app.

2. Gold-backed credit. Once a user has an accumulated gold balance sitting with Jar, offering a small, instant loan against that balance (or a gold-backed credit line) is a natural extension - the collateral already exists, the user is already KYC'd, and it solves a real short-term liquidity need without asking the user to sell an asset they've been deliberately building. This is a genuinely differentiated offering, since most lenders can't underwrite this cheaply or this fast because they don't already hold the collateral.

3. Insurance and protection products, sold in the same small-ticket language. Micro-insurance (accident cover, small health top-ups) framed the same way as the savings product - "₹10 a day" rather than "an annual premium" - would fit the same psychological model that made gold savings work. Jar wouldn't need to be the insurer; a distribution tie-up with an existing insurer, wrapped in Jar's UX and auto-debit rails, is enough to test demand cheaply.

4. A family/group savings layer built around real-life goals. Buddy savings already exists in a basic form; extending it toward specific occasions - a family wedding fund, a child's education fund, a group gift - with visible milestones and light social pressure (in a positive sense: a shared goal others can see progress on) plays directly to how gold-buying already works culturally in India, and gives the app a reason to be opened for something other than "did my auto-save go through today."

5. Data-backed credit scoring as a longer-term moat. This one's more speculative, but a user with a year or more of consistent, automated saving behavior is showing something a bureau score doesn't capture - discipline and consistency, even at small amounts. Over time, that saving history could become a legitimate underwriting signal Jar could use itself (for point 2 above) or potentially offer as an alternative-data input to partner lenders, for users who are otherwise thin-file or new-to-credit. This is the kind of thing that takes years of consistent data to be trustworthy, so it's worth starting to think about even though it isn't a near-term launch.

The common thread across all of these is that none of them ask Jar to abandon what it's good at - small ticket sizes, automation that requires no ongoing decision from the user, and a lot of accumulated trust from a product that's already proven itself with real money. The risk to avoid is trying to become a full-blown financial supermarket too quickly and diluting the one thing that made the first product work: it did one small thing extremely well before asking for anything else.